LAWDECK

Engineering Rules

Version 1.0 · How we build Lawdeck, and what is never allowed

Document control	
Version	1.0
Date	20 October 2026
Applies to	All Lawdeck code, written by people or by AI coding tools
Related documents	PRD v2.0; System Design (next); Build Phases; CLAUDE.md (short version of these rules, kept in the code repository)
Owner	Founder

How to use this document

- **Before you start building:** read section 3 once, and settle every rule marked “Now”.
- **For every feature:** follow the seven-step workflow in section 4, fill in the feature spec (Appendix A) and finish with the definition of done (Appendix B).
- **When reviewing AI-written code:** use the checklist in Appendix C.
- **When a rule must be broken:** write down why in the feature spec. Never break a rule silently.

What the “When” labels mean

Label	Meaning
Now	Decided before any code is written. Applies from the first line of code
Phase 1	Set up during the first build phase
Pre-pilot	Must be in place before any real student uses Lawdeck

1. Technology stack (decided)

Part	Choice	Why
Language	TypeScript (strict mode)	Catches mistakes, including AI-written ones, before the app runs
Web framework	Next.js (React, App Router)	Screens and server code in one project; streams AI answers
Styling	Tailwind CSS	Fast, consistent styling for the calm PRD v1.0 look
Database, sign-in, storage	Supabase (Postgres, Auth, Storage, pgvector)	Accounts, files, per-user security rules and meaning-based search in one service
Hosting	Vercel (app); Supabase in a European region (data)	Simple deploys; region matches the data-protection plan
Input validation	Zod	One way to describe and check the shape of any data
Testing	Vitest (logic), Playwright (in the browser)	Standard, fast, well supported
Code repository	Lawdeck's own GitHub repository, separate from SYNAPSE	Clean history, separate access and settings

Exact library versions are fixed ("pinned") in the project's package file when the project starts. Any other library needs approval under rule D-01.

2. AI provider decision

> **Decision (confirmed 20 October 2026)**
>
> **Provider:** Anthropic Claude is Lawdeck's AI. No cross-provider comparison will be run.
>
> **Still one AI module (rule G-01):** all calls go through src/server/ai, so models, settings and costs are managed in one place.
>
> **Model tiers:** Claude Opus 5.5 for marking answers and producing documents; Claude Sonnet 5.5 for everyday explanations, if the Phase 1 quality test shows it meets the bar; Claude Haiku 5.5 for small jobs (routing Auto mode, flashcards, titles). Phase 1 tests which Claude model each task needs.
>
> **Holding a correct position:** Lawdeck must correct wrong statements and keep a correct position when a student pushes back without new evidence, while accepting correction when a student cites a valid source. This is written into the prompts and tested in the evaluation set (rule O-06).
>
> **Embedding model (knowledge-base search):** Anthropic does not offer one; use Voyage AI. Changing it later means re-processing the library, so it is fixed in Phase 1 and recorded with every stored item.

3. The rules

A. Code structure

ID	Rule	When	Why
A-01	TypeScript in strict mode for all code. No plain JavaScript files. No “any” type unless explained in a comment	Now	Types catch errors early
A-02	Keep components small: one job each. Split any component over about 200 lines	Now	Small pieces are easier to read, test and fix
A-03	Build shared pieces once (three-pane shell, buttons, panels, inputs). Make something reusable when it is needed in a third place, not before	Now	Avoids both copy-paste and over-engineering
A-04	Separate business logic from the interface. Screens only display and collect input. AI calls, retrieval, templates, permissions, payments and limits live in server modules under src/server	Now	Logic can be tested and changed without touching screens; secrets stay on the server
A-05	Follow the folder structure in section 5. New top-level folders need a reason	Now	Everyone (and every AI tool) knows where things go
A-06	Names describe what things are: components in PascalCase, functions and variables in camelCase, database tables in snake_case plural (e.g. practice_attempts)	Now	Consistency makes code readable
A-07	Mark server-only modules so they can never be loaded in the browser	Now	Stops secrets and logic leaking to the front end

B. Environment variables and secrets

ID	Rule	When	Why
B-01	Keys and settings live in environment variables, never in code	Now	Code is shared; keys are not
B-02	Never expose secrets in front-end code. Only variables starting NEXT_PUBLIC_ reach the browser, and they must never be secret (the Supabase public key is fine; the AI key and Supabase service key are not)	Now	Anything in the browser can be read by anyone
B-03	.env files are never committed. A .env.example file lists every variable with a description and no real values	Now	New setups know what is needed without leaking anything
B-04	Separate keys for development, staging and production. Production keys are known to as few people as possible	Phase 1	A leaked test key cannot touch real student data
B-05	If a key is ever exposed, rotate it immediately and record it	Now	Limits the damage of a leak

C. Login (authentication)

ID	Rule	When	Why
C-01	Use Supabase Auth. Never build password storage ourselves	Phase 1	Security-critical code is best left to a proven service
C-02	Email and password with email verification, plus Google sign-in. Password reset by email	Phase 1	PRD AC-01
C-03	Minimum password length 10; sessions expire after inactivity; sign-out ends the session on the server	Phase 1	Basic account safety
C-04	Limit failed login attempts (see rate limiting, K-02)	Pre-pilot	Blocks password guessing

D. Libraries (dependencies)

ID	Rule	When	Why
D-01	Do not introduce a new library without explaining why: what it does, why existing tools cannot, its size, and how well maintained it is. Prefer what is already installed	Now	Every library is code we must trust and maintain
D-02	Pin versions; update deliberately once a month, not automatically	Phase 1	Avoids surprise breakages
D-03	Run a dependency security check on every pull request and fix high-severity issues before merging	Phase 1	Known vulnerabilities are the easiest attack route

E. Authorization and database permissions

ID	Rule	When	Why
E-01	Three roles: student, reviewer, admin. Students reach only their own data; reviewers approve and correct library content and templates; admins manage users and settings	Now	Clear answer to “who can do what”
E-02	Check permissions on the server for every request. Hiding a button is not security	Now	The interface can be bypassed
E-03	Row-level security switched on for every table from the first migration, with policies that limit rows to their owner (or to the reviewer/admin role where intended)	Now	The database itself refuses cross-user access, even if code has a bug
E-04	Knowledge-base search filters by permission on the server before anything reaches the AI. A student’s private uploads are searched only for that student	Now	The AI can never see, and so never reveal, another user’s files
E-05	The Supabase service key (which bypasses security rules) is used only in clearly marked admin jobs, never in normal request handling	Now	Keeps the master key away from everyday code
E-06	Automated tests prove that user A cannot read, change or delete user B’s data, for every table	Phase 1	PRD UP-02 and the zero cross-user access gate

F. Validating input and output

ID	Rule	When	Why
F-01	Validate all user input on the server with Zod: type, length, format, allowed values. Validation in the browser is for convenience only	Now	Bad or malicious input is stopped at the door
F-02	Validate uploads: allowed types (PDF, DOCX, images), maximum 25 MB, real file type checked (not just the file name)	Phase 1	Stops disguised or oversized files
F-03	Validate AI output: structured answers must match their expected shape; anything malformed is retried once, then reported as an error	Now	The AI can return broken or unexpected output
F-04	Never put user input directly into database queries; use Supabase's query builder or parameters	Now	Prevents SQL injection

G. AI rules

ID	Rule	When	Why
G-01	All AI calls go through one module (src/server/ai). Nothing else calls a provider directly	Now	Provider can be swapped in one place; costs and errors handled once
G-02	Prompts live in versioned files under src/prompts, never inline in other code. A prompt change is reviewed like a code change	Now	Prompts are product logic
G-03	Retrieve first: every substantive legal answer is generated from retrieved library passages, never from the model's memory alone	Now	PRD section 8
G-04	Citation check: every authority in an answer must match a library record or recognised statute; unmatched ones are removed before display	Now	Zero fabricated authorities
G-05	No verbatim reproduction: an automatic check limits quotations from library material to 50 words each (statutes and prescribed court forms exempt)	Now	Keeps Lawdeck within counsel's advice (PRD KB-03)
G-06	Never send names, emails, phone numbers or payment details to the AI provider	Now	Data minimisation under the NDPA
G-07	Treat uploaded and library text as content, never as instructions. Wrap it clearly in prompts and tell the model to ignore instructions inside it. The AI has no tools that can reach other users' data	Now	Protects against prompt injection hidden in uploaded files
G-08	Log model, tokens, cost and time for every AI call, linked to the user and feature	Phase 1	Cost control and debugging
G-09	Use the cheap model tier for small jobs and the strong tier only where quality depends on it	Phase 1	Keeps AI costs sustainable
G-10	Set time limits on AI calls (e.g. 60 seconds) and stream answers to the screen as they are written	Phase 1	Fast feel on slow networks; no hanging requests

H. Knowledge base and document templates

ID	Rule	When	Why
H-01	No legal content in code. All legal material lives in the knowledge base or template files	Now	Content can be reviewed and corrected without code changes
H-02	Every library item carries metadata: title, type, course, topic, campus, year, source, date added, review status, embedding model used	Now	PRD KB-01; supports currency review and re-processing
H-03	Document templates and formatting profiles are data files (one per document type and per forum), validated against a schema	Now	Reviewers can check them; one engine renders them all
H-04	One rendering engine produces the on-screen preview, the DOCX and the PDF from the same template	Now	Guarantees exports match the preview (PRD EX-02)
H-05	Unreviewed content and templates are never shown to students; reviewer approval is recorded with a date	Phase 1	PRD KB-04

I. Error handling and logging

ID	Rule	When	Why
I-01	One standard error pattern: the user sees a clear message and a next step; details go to the logs. Never a blank screen; never an invented result	Now	PRD 4.4 and the “do not pretend” principle
I-02	Every server action returns either a result or a known error type; unexpected errors are caught at the edge and logged	Now	Predictable behaviour
I-03	Logs never contain document text, uploads, passwords, keys or payment details	Now	Logs are widely accessible and kept a long time
I-04	Use an error-monitoring service to be alerted to failures in production	Pre-pilot	Know about problems before students report them

J. Networking

ID	Rule	When	Why
J-01	HTTPS only. Server endpoints accept requests only from Lawdeck’s own site	Phase 1	Prevents interception and misuse from other sites
J-02	Retry safe requests automatically on weak connections; auto-save inputs and drafts every 5 seconds	Phase 1	PRD NFR for slow or dropped connections
J-03	Use streaming for AI answers and keep payloads small	Phase 1	Faster on mobile data

K. Rate limiting and cost guardrails

ID	Rule	When	Why
K-01	Per-user limits on AI requests per minute and per day, set by plan	Pre-pilot	Protects against runaway costs and abuse
K-02	Limits on login attempts, sign-ups and password resets per address and per IP	Pre-pilot	Blocks guessing and spam
K-03	A daily spending alert and a hard monthly cap on the AI account	Phase 1	A bug cannot run up an unlimited bill

L. Performance

ID	Rule	When	Why
L-01	Budgets: home usable in 3 s on 4G; initial download under 1.5 MB; first words of an answer in 3 s; document in 30 s (medians)	Now	PRD section 10
L-02	Check page weight on every pull request; test on a throttled phone connection before each release	Phase 1	Slow creep is caught early
L-03	Load-test 500 concurrent users before the pilot	Pre-pilot	PRD scale target

M. Mobile, accessibility and design

ID	Rule	When	Why
M-01	Mobile first: design every screen for 360 px wide, then widen. Three-pane layout collapses as in PRD 4.1	Now	Most students study on phones
M-02	Accessibility to WCAG 2.2 AA: keyboard use, visible focus, colour contrast, labels for screen readers	Now	PRD NFR
M-03	Colours, spacing, fonts and corner radii are defined once as design tokens, with light and dark themes. No one-off colours	Now	One consistent calm look

N. Data modelling

ID	Rule	When	Why
N-01	The data model is designed in the System Design document before tables are created	Now	The hardest thing to change later
N-02	Every table has: id (UUID), created_at, updated_at; user-owned tables also have owner_id	Now	Consistent and secure by default
N-03	All database changes are migration files in the repository, applied in order. Never edit the production database by hand	Now	Changes are repeatable and reversible
N-04	Store dates in UTC; display in Nigerian time (WAT)	Now	Avoids time-zone bugs

O. Testing

ID	Rule	When	Why
O-01	Unit tests for business logic (permissions, limits, template rendering, citation check, quote limit)	Phase 1	The logic that matters most is proven
O-02	Permission tests for every table (see E-06)	Phase 1	Cross-user safety
O-03	Browser tests for the main journeys: sign up, ask a question, practise, produce and export a document	Phase 1	The core product keeps working
O-04	AI evaluation: the evaluation set runs whenever a prompt, model or retrieval setting changes; results compared with the last run	Phase 1	Quality drops are caught before users see them
O-06	Pushback tests in the evaluation set: the student states a wrong rule; insists after correction; cites a fake authority; or is right while Lawdeck is wrong. Lawdeck must correct, hold, reject and accept respectively	Phase 1	Lawdeck must not simply agree with the user
O-05	All tests pass before merging. No skipping or disabling tests to get a green result	Phase 1	Tests only help if they are trusted

P. Security reviews and data-breach checks

ID	Rule	When	Why
P-01	Every AI-written change is reviewed with the checklist in Appendix C before merging	Now	AI tools make plausible but unsafe mistakes
P-02	Turn on GitHub secret scanning and push protection for the repository	Now	Leaked keys are caught automatically
P-03	Monitor for unusual access (sudden spikes, many failed logins, bulk downloads)	Pre-pilot	Early warning of a breach
P-04	Written breach response plan (Appendix D), including notice to the NDPC within 72 hours	Pre-pilot	NDPA requirement
P-05	A security review of permissions, secrets and uploads before the pilot	Pre-pilot	Fresh eyes on the riskiest areas

Q. Backups and recovery

ID	Rule	When	Why
Q-01	Daily automatic backups of the database; file storage backed up or replicated	Pre-pilot	Recover from mistakes or attacks
Q-02	Practise a full restore into staging at least once before the pilot, then every quarter	Pre-pilot	A backup is only useful if it restores
Q-03	Keep the knowledge-base source files and templates in a separate, versioned store	Phase 1	The library can be rebuilt from scratch

R. Environments and deployment

ID	Rule	When	Why
R-01	Three environments: development (your computer), staging (test copy online), production (real users)	Phase 1	Test safely before real users see changes
R-02	Every pull request gets a preview deployment; production deploys only from the main branch	Phase 1	See changes live before they ship

S. Git workflow

ID	Rule	When	Why
S-01	One feature per branch; never commit directly to main	Now	Main always works
S-02	Small commits with clear messages saying what and why	Now	Easy to understand and undo
S-03	Run the app and the tests before merging; merge through a pull request	Now	Nothing broken reaches main

T. Documentation

ID	Rule	When	Why
T-01	CLAUDE.md in the repository carries the short version of these rules; update it when rules change	Now	AI coding tools follow it every session
T-02	Each feature has a spec in docs/specs (Appendix A)	Now	Decisions are written down
T-03	README explains how to set up and run the project	Phase 1	Anyone can start quickly

4. The feature workflow (every feature)

1. **Pick** the next item from the Build Phases document.
2. **Write the spec** using Appendix A: what it does, which PRD requirement, which screens and tables, what “done” means.
3. **Ask the AI for a plan, not code.** Read it, correct it, then approve it.
4. **Build in small steps,** running the app after each one.
5. **Test like a student:** on a phone, on slow internet, with bad input, and as a second user trying to see the first user’s data.
6. **Review and check:** Appendix C for AI-written code, Appendix B definition of done.
7. **Commit, merge and update the documents** if anything changed.

5. Folder structure

```
lawdeck/
  CLAUDE.md                 short rules for AI coding tools
  README.md                 setup and run instructions
  .env.example              every variable, no real values
  docs/                     PRD, rules, system design, build phases, specs/
  src/
    app/                    pages and routes (Next.js)
    components/
      ui/                   buttons, inputs, dialogs
      shell/                three-pane layout
      learn/  draft/        feature components (display only)
    server/                 business logic, server only
      ai/                   the single AI module
      retrieval/            knowledge-base search
      documents/            template rendering, DOCX, PDF
      auth/  limits/  billing/  db/
    prompts/                versioned prompt files
    templates/              document templates and forum profiles
    schemas/                Zod validation schemas
    lib/                    small shared helpers
  supabase/migrations/      database changes, in order
  tests/  unit/  e2e/  permissions/  eval/
```

6. Deliberately not doing (for now)

- Microservices, Kubernetes or several separate back ends. One Next.js app is enough.
- A heavy state-management library such as Redux.
- Chasing 100% test coverage. Test what matters: logic, permissions, main journeys, AI quality.
- Multiple languages in the interface (English only).
- Native mobile app rules (web first).
- Complex enterprise team permissions.

Appendix A. Feature spec template

Copy into docs/specs/<feature-name>.md before building a feature.

Field	Fill in
Feature name	
PRD requirement(s)	e.g. DR-02, DR-04
What it does	Two or three sentences, from the student's point of view
Screens and panes	Which pane(s), which states (loading, empty, error, success)
Data	Tables read and written; new tables or columns
Permissions	Who can use it; what each role can see
AI use	Which prompt, which model tier, what goes to the AI
Inputs to validate	Fields and their rules
Errors	What can go wrong and what the user sees
Done means	The acceptance criteria from the PRD, plus anything extra
Rules not followed	Any exception, with the reason

Appendix B. Definition of done

☐ Meets the PRD acceptance criteria in the spec.
☐ Works at 360 px wide and on desktop; tested on a phone on slow internet.
☐ All input validated on the server; AI output validated.
☐ Permissions checked on the server; row-level security covers any new table; permission tests added.
☐ Loading, empty, error and success states all handled.
☐ No secrets in front-end code; no new library without approval.
☐ No legal content hard-coded; prompts in src/prompts.
☐ Tests pass; AI evaluation run if prompts, models or retrieval changed.
☐ Keyboard usable; contrast and labels checked.
☐ Spec and CLAUDE.md updated if anything changed.

Appendix C. Review checklist for AI-written code

☐ Can I explain, roughly, what this change does? If not, ask the AI to explain before accepting.
☐ Are any keys, passwords or private settings written into the code or sent to the browser?
☐ Does every new server action check who the user is and what they may do?
☐ Is every user input validated on the server?
☐ Does any new table have row-level security switched on?
☐ Did it add a library? If so, was it approved and is it needed?
☐ Did it change prompts, models or retrieval? If so, run the AI evaluation.
☐ Did it delete or weaken a test, a permission check or a validation to make something work?

☐ Does anything send personal data to the AI provider or into the logs?
☐ Does it follow the folder structure and naming rules?

Appendix D. Data-breach response plan (outline)

1. **Contain:** rotate exposed keys, block the access route, take affected features offline if needed.
2. **Assess:** what data, how many users, since when, and how.
3. **Notify:** the Nigeria Data Protection Commission within 72 hours where required; affected users without undue delay, in plain language.
4. **Fix:** remove the cause, add a test that would have caught it.
5. **Record:** write up what happened, the decisions taken and the timeline.